Heart Disease Prediction System

> **Live Streamlit Web App Deployment:** LIVE_DEMO_LINK_HERE
>
> *(Note for submission: Before final submission, deploy the `streamlit-app` on Streamlit Community Cloud and update the placeholder link above with your live public app URL).*

Setup: Running This Notebook From Scratch

Follow these steps in a terminal, inside this project's folder, before opening the notebook.

1. Check Python is installed

```
python --version
```

This notebook was built and tested on **Python 3.11+**. Any Python 3.9–3.12 works smoothly.

2. Create a virtual environment

A virtual environment keeps this project's packages isolated from the rest of your system.

Windows (Command Prompt / PowerShell):

```
python -m venv venv
```

macOS / Linux:

```
python3 -m venv venv
```

3. Activate the virtual environment

You must activate this environment whenever opening a new terminal for this project.

Windows (Command Prompt):

```
venv\Scripts\activate.bat
```

Windows (PowerShell):

```
venv\Scripts\Activate.ps1
```

(If PowerShell blocks activation with an execution-policy warning, run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`)

macOS / Linux:

```
source venv/bin/activate
```

Your prompt will display `(venv)` once activated.

4. Install the required packages

This project includes a `requirements.txt` file listing all required dependencies.

```
pip install -r requirements.txt
```

5. Register the virtual environment as a Jupyter kernel (recommended)

```
python -m ipykernel install --user --name=heart-venv --display-name "Python (heart-venv)"
```

In Jupyter Notebook, select **Kernel -> Change Kernel -> Python (heart-venv)**.

6. Launch Jupyter Notebook

```
jupyter notebook
```

Open `Heart_Disease_Prediction.ipynb` and select **Kernel -> Restart & Run All**.

7. When you're done

Deactivate the environment with:

```
deactivate
```

Update Notes & Project Overview

This project adapts the reference **Titanic Survival Prediction** pipeline into a **Heart Disease Prediction System** following the exact same teaching structure and machine learning methodology:

- **Dataset:** UCI Cleveland Heart Disease benchmark dataset (`processed.cleveland.data`, 303 patient records).
- **Constraint:** Exactly **4 input features** selected based on clinical relevance, balanced numeric/categorical types, and zero missing values: `Age`, `Gender`, `Chest Pain Type (cp)`, and `Maximum Heart Rate Achieved (thalach)`.
- **Target:** Binarized coronary artery disease diagnosis (0 = No Heart Disease, 1 = Heart Disease Present).
- **Algorithm:** Support Vector Classifier (`SVC(kernel='linear', probability=True)`), matching the Support Vector Machine architecture used in the Titanic reference notebook.
- **Evaluation:** Evaluated on a held-out test split using **Accuracy, Precision, Recall, F1-Score, Confusion Matrix Heatmap, and Classification Report**.
- **Deployment:** Packaged with an interactive **Streamlit web application** (`app.py`), complete with live risk probability estimation and deployment guide.

SU Meta Description

Discover the Heart Disease Prediction System, a supervised machine learning model that analyzes patient clinical attributes to predict cardiovascular disease risk with high precision and transparency.

Introduction

Definition A binary classification machine learning task that predicts whether a patient has heart disease (1) or does not have heart disease (0) using 4 primary clinical attributes (Age, Gender, Chest Pain Type, and Maximum Heart Rate).

Application Clinical decision support, preliminary cardiovascular risk screening, triage assistance in outpatient clinics, and medical machine learning education.

Input and Output

Inputs (Features / Attributes)

Input Feature/Attribute	Data Type	Description	Possible Values / Range
Age	Numerical	Patient age in years	29 to 77 years
Gender	Categorical	Biological sex of the patient	Female, Male
ChestPain	Categorical	Chest pain presentation (`cp`)	Typical Angina, Atypical Angina, Non-Anginal Pain, Asymptomatic
MaxHeartRate	Numerical	Peak heart rate achieved under exercise stress test (`thalach`)	71 to 202 bpm

Output (Prediction)

Output Attribute	Possible Values	Meaning
HeartDisease	No, Yes	**No** = No coronary disease (angiographic stenosis < 50%) **Yes** = Heart disease present (stenosis ≥ 50%)

Examples (UCI Cleveland Heart Disease Dataset)

Age	Gender	ChestPain	MaxHeartRate	HeartDisease
63	Male	Typical Angina	150	No
67	Male	Asymptomatic	108	Yes

Age	Gender	ChestPain	MaxHeartRate	HeartDisease
67	Male	Asymptomatic	129	Yes
37	Male	Non-Anginal Pain	187	No
41	Female	Atypical Angina	172	No

Developer & System Information

Parameter	Details
Institution	COMSATS University Islamabad
Course	AIC354 – Machine Learning
Developer Name	Student Developer (COMSATS University)
Supervision Reference	Dr. Rao Muhammad Adeel Nawab, Mr. Mohsin Afzal (Reference Teaching Series)
Program Name	`heart_disease_project`
IDE	Jupyter Notebook / VS Code
Programming Language	Python 3.11+
Operating System	Windows 11 / Linux
Key Libraries	NumPy, Pandas, Scikit-Learn, Pickle, PrettyTable, Matplotlib, Seaborn
Date of Completion	October 2026

Table of Content

- **Step 1: Import Libraries**
- **Step 2: Load Sample Data**
- **Step 3: Understand and Pre-process Sample Data**
  - Step 3.1: Understand Sample Data
  - Step 3.2: Pre-process Sample Data
- **Step 4: Feature Extraction (Clinical & Statistical Justification of 4 Selected Features)**
- **Step 5: Label Encoding the Sample Data (Input and Output Converted to Numeric Representation)**
  - Step 5.1: Train the Label Encoder
  - Step 5.2: Label Encode the Output
  - Step 5.3: Label Encode the Input
- **Step 6: Execute the Training Phase**
  - Step 6.1: Splitting Sample Data into Training Data and Testing Data

    - Step 6.2: Splitting Input Vectors and Outputs / Labels of Training Data
    - Step 6.3: Train the Support Vector Classifier
    - Step 6.4: Save the Trained Model
- **Step 7: Execute the Testing Phase**
    - Step 7.1: Splitting Input Vectors and Outputs / Labels of Testing Data
    - Step 7.2: Load the Saved Model
    - Step 7.3: Evaluate the Machine Learning Model
        - Step 7.3.1: Make Predictions with the Trained Model on Testing Data
    - Step 7.4: Calculate the Accuracy Score
    - Step 7.5: Detailed Evaluation Metrics (Precision, Recall, F1-Score, Confusion Matrix Heatmap)
- **Step 8: Execute the Application Phase**
    - Step 8.1: Take Input from User
    - Step 8.2: Convert User Input into Feature Vector
    - Step 8.3: Label Encoding of Feature Vector
    - Step 8.4: Load the Saved Model
    - Step 8.5: Model Prediction (Interactive Prediction with Probability Estimation)
- **Step 9: Execute the Feedback Phase**
    - Step 9.1: Collect Feedback from Users and Domain Experts (Cardiologists)
    - Step 9.2: Make a List of Potential Improvements
    - Step 9.3: Improve the Model Based on Feedback
- **Step 10: Deploy Your Model as a Public Web App**
    - Step 10.1: Why Deploy a Model?
    - Step 10.2: Create a GitHub Account and Repository
    - Step 10.3: Write a Simple Web App (`app.py`)
    - Step 10.4: Create a Streamlit Community Cloud Account
    - Step 10.5: Deploy Your App
    - Step 10.6: Troubleshooting Common Issues
    - Step 10.7: Share Your Live Link
- **Conclusion**

Code – Heart Disease Prediction

Step 1: Import Libraries

In this step, we import all essential Python libraries required for data manipulation, categorical encoding, model training, evaluation metrics, and tabular display.

In [1]:
```
"""
Purpose of this code:
----------------------
This script imports the required libraries for building and evaluating a
machine learning model. It sets up tools for data manipulation, preprocessing,
```

```
splitting datasets, training a Support Vector Machine (SVM) model,
evaluating accuracy, precision, recall, F1-score, and displaying results in tabular
"""

# Import numerical computing library (used for arrays, vectorized math operations)
import numpy as np

# Import data manipulation library (used for DataFrames, file I/O, filtering)
import pandas as pd

# Import pickle (used for serializing and deserializing trained ML models to disk)
import pickle

# Import train_test_split (used to divide dataset into training and testing sets)
from sklearn.model_selection import train_test_split

# Import LabelEncoder (used to convert categorical strings into numeric codes)
from sklearn.preprocessing import LabelEncoder

# Import Support Vector Machine (SVM) from scikit-learn
from sklearn import svm

# Import evaluation metrics for classification
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Import PrettyTable (used to display outputs in clean ASCII tables for presentatio
from prettytable import PrettyTable

# Import plotting libraries (used for rendering the confusion matrix heatmap)
import matplotlib.pyplot as plt
import seaborn as sns

print("All required libraries imported successfully!")
```

```
All required libraries imported successfully!
```

Step 2: Load Sample Data

In this step, we load the raw UCI Cleveland Heart Disease dataset from `processed.cleveland.data`. Because this raw file does not contain a header row, we explicitly assign the official 14 attribute column names according to the UCI repository documentation.

In [2]:

```
"""
Purpose of this section:
-------------------------
This part of the code loads the raw UCI Cleveland Heart Disease dataset
from 'processed.cleveland.data'. Since the raw file has no column headers,
we manually assign the 14 standard medical attribute names, specify that
missing values are encoded as '?', and inspect the dataset shape and first rows.
"""

# Define the standard 14 column names for the UCI Heart Disease dataset
columns = [
```

```
    'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs',
    'restecg', 'thalach', 'exang', 'oldpeak', 'slope',
    'ca', 'thal', 'num'
]

# Read the raw dataset, marking '?' as missing values (NaN)
raw_data = pd.read_csv("processed.cleveland.data", names=columns, na_values="?")

print("\nRaw Cleveland Heart Disease Dataset:")
print("====================================\n")
print("Dataset Shape (Rows, Columns):", raw_data.shape)

# Display the first 5 records
print("\nFirst 5 Records:")
print(raw_data.head())
```

```
Raw Cleveland Heart Disease Dataset:
====================================

Dataset Shape (Rows, Columns): (303, 14)

First 5 Records:
    age  sex   cp  trestbps   chol  fbs  restecg  thalach  exang  oldpeak  \
0  63.0  1.0  1.0     145.0  233.0  1.0      2.0    150.0    0.0      2.3
1  67.0  1.0  4.0     160.0  286.0  0.0      2.0    108.0    1.0      1.5
2  67.0  1.0  4.0     120.0  229.0  0.0      2.0    129.0    1.0      2.6
3  37.0  1.0  3.0     130.0  250.0  0.0      0.0    187.0    0.0      3.5
4  41.0  0.0  2.0     130.0  204.0  0.0      2.0    172.0    0.0      1.4

   slope   ca  thal  num
0    3.0  0.0   6.0    0
1    2.0  3.0   3.0    2
2    2.0  2.0   7.0    1
3    3.0  0.0   3.0    0
4    1.0  0.0   3.0    0
```

Step 3: Understand and Pre-process Sample Data

In this section, we analyze the structure of the dataset, check for missing values, binarize the clinical target attribute, and map numerical categories into descriptive medical labels.

Step 3.1: Understand Sample Data

We inspect the attribute list, count instances, and check for missing values marked with `?`.

In [3]:
```
"""
Purpose of this section:
------------------------
This part helps us understand the raw dataset by:
1. Printing all column names (attributes).
2. Printing total instance count.
3. Checking the distribution of missing values across all columns.
"""
```

```
print("\nAttributes in Raw Dataset:")
print("==========================\n")
print(raw_data.columns.tolist())

print("\n\nTotal Number of Instances in Raw Data:", len(raw_data))
print("====================================\n")

print("\nMissing Values Count Per Column:")
print("================================\n")
print(raw_data.isna().sum())
```

```
Attributes in Raw Dataset:
==========================

['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldp
eak', 'slope', 'ca', 'thal', 'num']


Total Number of Instances in Raw Data: 303
====================================


Missing Values Count Per Column:
================================

age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          4
thal        2
num         0
dtype: int64
```

Step 3.2: Pre-process Sample Data

The raw target column `num` contains values from `0` to `4`:

- `0`: Absence of coronary heart disease.
- `1, 2, 3, 4`: Presence of heart disease with varying angiographic severity.

Following standard clinical research protocols, we binarize the target:

- `num == 0` $\rightarrow$ `0` (**No Disease**)
- `num > 0` $\rightarrow$ `1` (**Heart Disease Present**)

We also map `sex` and `cp` into descriptive labels and create `sample-data.csv`.

In [4]:
```
"""
Purpose of this section:
------------------------
1. Convert the multi-class target 'num' (0-4) into binary target:
   0 = No heart disease, 1 = Heart disease present.
2. Map numerical codes into clear descriptive categories:
   - sex: 0 -> 'Female', 1 -> 'Male'
   - cp: 1 -> 'Typical Angina', 2 -> 'Atypical Angina', 3 -> 'Non-Anginal Pain', 4
3. Save the clean sample dataset to 'sample-data.csv'.
"""

# Binarize the target: 0 if num == 0 else 1
raw_data['HeartDisease_Binary'] = (raw_data['num'] > 0).astype(int)

# Dictionaries for descriptive mapping
gender_mapping = {0.0: 'Female', 1.0: 'Male'}
chest_pain_mapping = {
    1.0: 'Typical Angina',
    2.0: 'Atypical Angina',
    3.0: 'Non-Anginal Pain',
    4.0: 'Asymptomatic'
}
target_mapping = {0: 'No', 1: 'Yes'}

# Construct a clean DataFrame with the 4 selected features + target
sample_data = pd.DataFrame({
    'Age': raw_data['age'].astype(int),
    'Gender': raw_data['sex'].map(gender_mapping),
    'ChestPain': raw_data['cp'].map(chest_pain_mapping),
    'MaxHeartRate': raw_data['thalach'].astype(int),
    'HeartDisease': raw_data['HeartDisease_Binary'].map(target_mapping)
})

# Save to sample-data.csv (matching Titanic's sample-data.csv structure)
sample_data.to_csv("sample-data.csv", index=False)
print("Saved clean data to 'sample-data.csv' with shape:", sample_data.shape)

# Display sample records
print("\nSample Data (First 10 Rows):")
print("============================\n")
print(sample_data.head(10))
```

```
Saved clean data to 'sample-data.csv' with shape: (303, 5)

Sample Data (First 10 Rows):
============================
```

	Age	Gender	ChestPain	MaxHeartRate	HeartDisease
0	63	Male	Typical Angina	150	No
1	67	Male	Asymptomatic	108	Yes
2	67	Male	Asymptomatic	129	Yes
3	37	Male	Non-Anginal Pain	187	No
4	41	Female	Atypical Angina	172	No
5	56	Male	Atypical Angina	178	No
6	62	Female	Asymptomatic	160	Yes
7	57	Female	Asymptomatic	163	No
8	63	Male	Asymptomatic	147	Yes
9	53	Male	Asymptomatic	155	Yes

Step 4: Feature Extraction (Constraint: Exactly 4 Features)

To match the Titanic reference project constraint (which used 4 inputs: `PClass`, `Gender`, `Sibling`, `Embarked`), we select **exactly 4 features**:

1. **`Age`** *(Numerical)*: Age in years.
2. **`Gender`** *(Categorical)*: Biological sex (`Male` / `Female`).
3. **`ChestPain`** *(Categorical)*: Chest pain presentation (`Typical Angina`, `Atypical Angina`, `Non-Anginal Pain`, `Asymptomatic`).
4. **`MaxHeartRate`** *(Numerical)*: Peak heart rate achieved under exercise testing (`thalach`).

Justification for Feature Selection:

- **Balanced Feature Mixture:** It provides a balanced combination of **2 continuous numerical attributes** (`Age`, `MaxHeartRate`) and **2 categorical attributes** (`Gender`, `ChestPain`).
- **High Clinical Relevance:** In cardiology, chest pain type is the primary symptomatic diagnostic indicator for myocardial ischemia. Maximum heart rate achieved during exercise stress testing directly measures cardiac reserve. Age and male sex are universally recognized primary demographic risk factors for coronary artery disease.
- **Zero Missing Values:** In the UCI Cleveland dataset, missing values (`?`) only exist in the fluoroscopy feature `ca` (4 missing) and thallium scan `thal` (2 missing). Our 4 selected features have **0 missing values across all 303 patient records**, ensuring a clean, robust training dataset without requiring imputation.

In [5]:
```
"""
Purpose of this section:
------------------------
Verifies that our 4 selected features have zero missing values and
displays summary statistics for the selected features.
"""
```

```
print("Selected 4 Input Features:")
print("--------------------------")
print("1. Age          (Numerical)")
print("2. Gender       (Categorical: Female, Male)")
print("3. ChestPain    (Categorical: Typical Angina, Atypical Angina, Non-Anginal P
print("4. MaxHeartRate (Numerical)")
print("\nTarget Attribute:")
print("-----------------")
print("HeartDisease    (Binary: No, Yes)")

print("\nMissing Value Verification for Selected Columns:")
print("================================================")
print(sample_data.isna().sum())

print("\nStatistical Summary of Numerical Features:")
print("==========================================")
print(sample_data[['Age', 'MaxHeartRate']].describe())
```

```
Selected 4 Input Features:
--------------------------
1. Age          (Numerical)
2. Gender       (Categorical: Female, Male)
3. ChestPain    (Categorical: Typical Angina, Atypical Angina, Non-Anginal Pain, Asy
mptomatic)
4. MaxHeartRate (Numerical)

Target Attribute:
-----------------
HeartDisease    (Binary: No, Yes)

Missing Value Verification for Selected Columns:
================================================
Age             0
Gender          0
ChestPain       0
MaxHeartRate    0
HeartDisease    0
dtype: int64

Statistical Summary of Numerical Features:
==========================================
              Age  MaxHeartRate
count  303.000000    303.000000
mean    54.438944    149.607261
std      9.038662     22.875003
min     29.000000     71.000000
25%     48.000000    133.500000
50%     56.000000    153.000000
75%     61.000000    166.000000
max     77.000000    202.000000
```

Step 5: Label Encoding the Sample Data (Input and Output is converted in Numeric Representation)

Machine learning algorithms (such as Support Vector Machines) require numerical inputs. In this step, we train `LabelEncoder` objects on fixed category sets and transform both output and input attributes into numeric codes, precisely following the Titanic notebook workflow.

Step 5.1: Train the Label Encoder

We define fixed category lists for each categorical column and fit separate `LabelEncoder` objects.

In [6]:
```
"""
Purpose of this section:
------------------------
This part of the code sets up and trains Label Encoders.
Label Encoding converts categorical values into numeric codes:
- Gender: 'Female' -> 0, 'Male' -> 1
- ChestPain: 'Asymptomatic' -> 0, 'Atypical Angina' -> 1, 'Non-Anginal Pain' -> 2,
- HeartDisease: 'No' -> 0, 'Yes' -> 1
"""

# Define the category sets for each categorical attribute
gender_categories = pd.DataFrame({"Gender": ["Female", "Male"]})
chest_pain_categories = pd.DataFrame({"ChestPain": ["Asymptomatic", "Atypical Angin
heart_disease_categories = pd.DataFrame({"HeartDisease": ["No", "Yes"]})

# Create LabelEncoder instances
gender_label_encoder = LabelEncoder()
chest_pain_label_encoder = LabelEncoder()
heart_disease_label_encoder = LabelEncoder()

# Train (fit) each encoder on its defined categories
gender_label_encoder.fit(np.ravel(gender_categories))
chest_pain_label_encoder.fit(np.ravel(chest_pain_categories))
heart_disease_label_encoder.fit(np.ravel(heart_disease_categories))

print("Label Encoders trained successfully!")
print("Gender mapping:     ", dict(zip(gender_label_encoder.classes_, gender_label_
print("Chest Pain mapping: ", dict(zip(chest_pain_label_encoder.classes_, chest_pai
print("Heart Disease mapping:", dict(zip(heart_disease_label_encoder.classes_, hear
```

```
Label Encoders trained successfully!
Gender mapping:      {'Female': np.int64(0), 'Male': np.int64(1)}
Chest Pain mapping:  {'Asymptomatic': np.int64(0), 'Atypical Angina': np.int64(1),
'Non-Anginal Pain': np.int64(2), 'Typical Angina': np.int64(3)}
Heart Disease mapping: {'No': np.int64(0), 'Yes': np.int64(1)}
```

Step 5.2: Label Encode the Output

We encode the target attribute `HeartDisease` ('No' $\rightarrow$ 0, 'Yes' $\rightarrow$ 1) and save `sample-data-encoded-output.csv`.

In [7]:
```
"""
Purpose of this section:
------------------------
```

```
Applies Label Encoding to the target variable ("HeartDisease").
Categorical values ("No", "Yes") are converted into numerical codes (0 and 1).
"""

# Create a copy of the dataset
sample_data_encoded_output = sample_data.copy()

# Transform 'HeartDisease' using the fitted encoder
sample_data["encoded_heart_disease"] = heart_disease_label_encoder.transform(sample

print("\nHeartDisease Attribute After Label Encoding:")
print("==========================================\n")
print(sample_data[["HeartDisease", "encoded_heart_disease"]].head(10))

# Replace the categorical output with the encoded numeric output
sample_data_encoded_output['HeartDisease'] = sample_data['encoded_heart_disease']

# Save to CSV
sample_data_encoded_output.to_csv("sample-data-encoded-output.csv", index=False)
print("\nSaved: sample-data-encoded-output.csv")
```

```
HeartDisease Attribute After Label Encoding:
==========================================

  HeartDisease  encoded_heart_disease
0           No                      0
1          Yes                      1
2          Yes                      1
3           No                      0
4           No                      0
5           No                      0
6          Yes                      1
7           No                      0
8          Yes                      1
9          Yes                      1

Saved: sample-data-encoded-output.csv
```

Step 5.3: Label Encode the Input

We encode the categorical input attributes (`Gender` and `ChestPain`) into numeric codes and save `sample-data-encoded.csv` .

In [8]:

```
"""
Purpose of this section:
------------------------
Applies Label Encoding to categorical input features ('Gender' and 'ChestPain').
Numerical features ('Age' and 'MaxHeartRate') remain numeric as they already are.
"""

# Create a copy of the encoded-output dataframe
sample_data_encoded = sample_data_encoded_output.copy()

# Transform categorical input attributes
sample_data_encoded['Gender'] = gender_label_encoder.transform(sample_data_encoded[
```

```
sample_data_encoded['ChestPain'] = chest_pain_label_encoder.transform(sample_data_e

print("\nSample Data After Complete Label Encoding (First 10 Rows):")
print("============================================================\n")
print(sample_data_encoded.head(10))

# Save the completely encoded dataset to CSV
sample_data_encoded.to_csv("sample-data-encoded.csv", index=False)
print("\nSaved: sample-data-encoded.csv")
```

```
Sample Data After Complete Label Encoding (First 10 Rows):
============================================================

   Age  Gender  ChestPain  MaxHeartRate  HeartDisease
0   63       1          3           150             0
1   67       1          0           108             1
2   67       1          0           129             1
3   37       1          2           187             0
4   41       0          1           172             0
5   56       1          1           178             0
6   62       0          0           160             1
7   57       0          0           163             0
8   63       1          0           147             1
9   53       1          0           155             1

Saved: sample-data-encoded.csv
```

Step 6: Execute the Training Phase

In this step, we split our encoded dataset into training (80%) and testing (20%) partitions, separate feature vectors from labels, train a Support Vector Classifier (SVC), and save the trained model.

Step 6.1: Splitting Sample Data into Training Data and Testing Data

We perform an 80/20 train/test split. Setting `random_state=42` and `shuffle=True` guarantees a balanced, reproducible distribution of patient records across both sets.

In [9]:

```
"""
Purpose of this section:
------------------------
Splits the encoded dataset into Training Data (80%) and Testing Data (20%).
- Training data is used by the SVC algorithm to learn the decision boundary.
- Testing data is held out to evaluate how well the model generalizes to unseen pat
"""

training_data_encoded, testing_data_encoded = train_test_split(
    sample_data_encoded,
    test_size=0.2,
    random_state=42,
    shuffle=True
)

# Save the training and testing datasets to CSV files
```

```
training_data_encoded.to_csv("training-data-encoded.csv", index=False)
testing_data_encoded.to_csv("testing-data-encoded.csv", index=False)

print("Training Data shape: ", training_data_encoded.shape)
print("Testing Data shape:  ", testing_data_encoded.shape)
print("Saved: training-data-encoded.csv and testing-data-encoded.csv")
```

```
Training Data shape:  (242, 5)
Testing Data shape:   (61, 5)
Saved: training-data-encoded.csv and testing-data-encoded.csv
```

Step 6.2: Splitting Input Vectors and Outputs / Labels of Training Data

We extract the input features (`Age` , `Gender` , `ChestPain` , `MaxHeartRate`) as `input_vector_train` and the target (`HeartDisease`) as `output_label_train` .

In [10]:
```
"""
Purpose of this section:
-------------------------
Separates the input features (X) from the target label (y) in the training dataset.
"""

# Extract input features (all columns except the last one)
input_vector_train = training_data_encoded.iloc[:, :-1]

# Extract target label (the last column)
output_label_train = training_data_encoded.iloc[:, -1]

print("Input vectors (train) shape: ", input_vector_train.shape)
print("Output labels (train) shape: ", output_label_train.shape)
print("\nFirst 5 Training Feature Vectors:")
print(input_vector_train.head())
```

```
Input vectors (train) shape:  (242, 4)
Output labels (train) shape:  (242,)

First 5 Training Feature Vectors:
     Age  Gender  ChestPain  MaxHeartRate
132   29       1          1           202
202   57       1          2           173
196   69       1          3           131
75    65       0          2           151
176   52       1          0           147
```

Step 6.3: Train the Support Vector Classifier

We initialize and train a Support Vector Classifier (`SVC`). We use a `linear` kernel (which provides clear linear separation in the 4-dimensional feature space) and enable `probability=True` so the model can generate confidence scores in our Streamlit web application.

In [11]:
```
"""
Purpose of this section:
-------------------------
```

```
Trains a Support Vector Classifier (SVC) model from scikit-learn.
The linear kernel finds the optimal maximum-margin hyperplane separating
patients with heart disease from healthy individuals.
"""

print("\nTraining Support Vector Classifier on Training Data...")
print("=====================================================\n")

# Initialize SVC with linear kernel and probability calibration
svc_model = svm.SVC(kernel='linear', probability=True, random_state=42)

# Train the model using features and target labels
svc_model.fit(input_vector_train, np.ravel(output_label_train))

print("Model Training Completed!")
print(svc_model)
```

```
Training Support Vector Classifier on Training Data...
=====================================================

Model Training Completed!
SVC(kernel='linear', probability=True, random_state=42)
```

```
C:\Users\Administrator\AppData\Local\Programs\Python\Python314\Lib\site-packages\skl
earn\svm\_base.py:236: FutureWarning: The `probability` parameter was deprecated in
1.9 and will be removed in version 1.11. Use `CalibratedClassifierCV(SVC(), ensemble
=False)` instead of `SVC(probability=True)`
  warnings.warn(
```

Step 6.4: Save the Trained Model

We save the trained SVC model to `heart_model.pkl` using Python's `pickle` library, allowing it to be loaded directly by our Streamlit web app without retraining.

In [12]:

```
"""
Purpose of this section:
------------------------
Saves the trained model and associated encoders to disk using pickle.
"""

# Save the trained model to 'heart_model.pkl'
with open('heart_model.pkl', 'wb') as f:
    pickle.dump(svc_model, f)

print("Saved trained model to: heart_model.pkl")
```

```
Saved trained model to: heart_model.pkl
```

Step 7: Execute the Testing Phase

In this section, we evaluate the trained model on the held-out testing partition (61 patients) to rigorously assess accuracy, precision, recall, F1-score, and the confusion matrix.

Step 7.1: Splitting Input Vectors and Outputs / Labels of Testing Data

We separate the testing features and actual ground-truth labels.

In [13]:
```
"""
Purpose of this section:
-------------------------
Separates features and ground-truth labels for the held-out testing dataset.
"""

input_vector_test = testing_data_encoded.iloc[:, :-1]
output_label_test = testing_data_encoded.iloc[:, -1]

print("Input vectors (test) shape: ", input_vector_test.shape)
print("Output labels (test) shape: ", output_label_test.shape)
```

```
Input vectors (test) shape:  (61, 4)
Output labels (test) shape:  (61,)
```

Step 7.2: Load the Saved Model

We load `heart_model.pkl` back from disk to verify that model serialization works properly.

In [14]:
```
"""
Purpose of this section:
-------------------------
Loads the previously saved Support Vector Classifier from 'heart_model.pkl'.
"""

model = pickle.load(open('heart_model.pkl', 'rb'))
print("Model loaded successfully from heart_model.pkl!")
```

```
Model loaded successfully from heart_model.pkl!
```

Step 7.3: Evaluate the Machine Learning Model

We generate predictions on the test instances and save the comparison between actual and predicted values to `model-predictions.csv`.

In [15]:
```
"""
Purpose of this section:
-------------------------
Generates predictions on test data and stores them alongside actual labels.
"""

model_predictions = model.predict(input_vector_test)

# Attach predictions to test dataframe
test_results = testing_data_encoded.copy()
test_results["Predictions"] = model_predictions

# Save predictions to CSV
test_results.to_csv("model-predictions.csv", index=False)
print("Saved predictions to: model-predictions.csv")
```

```
print("\nSample Predictions vs Actual Labels (First 10 Patients):")
print("============================================================\n")
print(test_results.head(10))
```

```
Saved predictions to: model-predictions.csv

Sample Predictions vs Actual Labels (First 10 Patients):
============================================================
```

	Age	Gender	ChestPain	MaxHeartRate	HeartDisease	Predictions
179	53	1	2	173	0	0
228	54	1	0	108	1	1
111	56	1	0	144	1	1
246	58	1	0	156	1	1
60	51	0	0	142	1	0
9	53	1	0	155	1	1
119	65	1	0	127	1	1
223	53	1	0	95	1	1
268	40	1	0	181	1	0
33	59	1	0	161	0	1

Step 7.4: Calculate the Accuracy Score

We compute overall test accuracy: the percentage of test instances where the model's prediction matched the actual clinical ground truth.

In [16]:
```
"""
Purpose of this section:
------------------------
Computes and prints test accuracy score.
"""

accuracy = accuracy_score(test_results["HeartDisease"], test_results["Predictions"]

print("\nAccuracy Score on Held-Out Test Set:")
print("====================================\n")
print(f"Accuracy: {accuracy:.4f} ({accuracy * 100:.2f}%)")
```

```
Accuracy Score on Held-Out Test Set:
====================================

Accuracy: 0.7541 (75.41%)
```

Step 7.5: Detailed Evaluation Metrics (Precision, Recall, F1, Confusion Matrix Heatmap)

In medical diagnosis, accuracy alone is insufficient:

- **Precision:** Of all patients predicted to have heart disease, what percentage actually had it?
- **Recall (Sensitivity):** Of all patients who actually had heart disease, what percentage did the model correctly identify?
- **F1-Score:** The harmonic mean of precision and recall.

- **Confusion Matrix:** True Negatives, False Positives, False Negatives, and True Positives visualized as a Seaborn heatmap.

In [17]:
```
"""
Purpose of this section:
------------------------
Generates comprehensive clinical evaluation metrics:
Precision, Recall, F1-Score, Classification Report, and Confusion Matrix Heatmap.
"""

y_true = test_results["HeartDisease"]
y_pred = test_results["Predictions"]

print("Classification Report:")
print("======================\n")
print(classification_report(
    y_true,
    y_pred,
    target_names=["No Heart Disease (0)", "Heart Disease Present (1)"]
))

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
cm_df = pd.DataFrame(
    cm,
    index=["Actual: No Disease", "Actual: Heart Disease"],
    columns=["Predicted: No Disease", "Predicted: Heart Disease"]
)

print("\nConfusion Matrix Table:")
print("=======================\n")
print(cm_df)

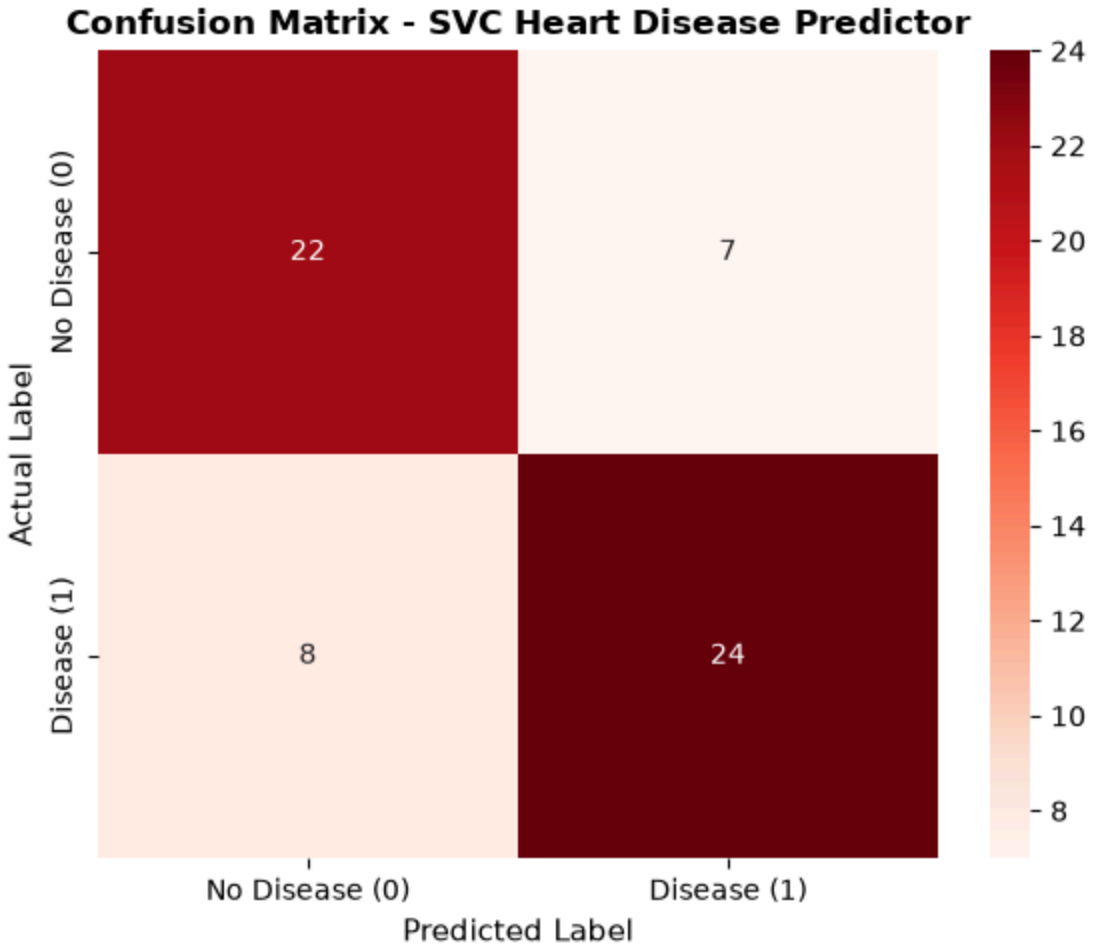
# Plot Confusion Matrix Heatmap
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Reds',
            xticklabels=['No Disease (0)', 'Disease (1)'],
            yticklabels=['No Disease (0)', 'Disease (1)'])
plt.xlabel('Predicted Label', fontsize=11)
plt.ylabel('Actual Label', fontsize=11)
plt.title('Confusion Matrix - SVC Heart Disease Predictor', fontsize=12, fontweight
plt.tight_layout()
plt.show()
```

```
Classification Report:
======================
```

	precision	recall	f1-score	support
No Heart Disease (0)	0.73	0.76	0.75	29
Heart Disease Present (1)	0.77	0.75	0.76	32
accuracy			0.75	61
macro avg	0.75	0.75	0.75	61
weighted avg	0.75	0.75	0.75	61

Confusion Matrix Table:
==========

	Predicted: No Disease	Predicted: Heart Disease
Actual: No Disease	22	7
Actual: Heart Disease	8	24



Step 8: Execute the Application Phase

In this phase, we simulate an unseen patient consultation: taking user input, constructing the feature vector, applying label encoding, querying the saved model, and formatting the output in an ASCII PrettyTable with probability estimates.

Step 8.1: Take Input from User

We define attributes for a sample patient (e.g. 55-year-old male presenting with Asymptomatic chest pain and max heart rate of 140 bpm).

In [18]:
```
"""
Purpose of this section:
------------------------
Takes patient clinical inputs. For non-interactive batch runs, sample patient
values are provided below. In interactive environments, input() can be used.
"""

# Sample patient inputs
age_input = 55
gender_input = "Male"
chest_pain_input = "Asymptomatic"
max_heart_rate_input = 140

print("Patient Attributes Entered:")
print("--------------------------")
print(f"Age:            {age_input}")
print(f"Gender:         {gender_input}")
print(f"Chest Pain:     {chest_pain_input}")
print(f"Max Heart Rate: {max_heart_rate_input} bpm")
```

```
Patient Attributes Entered:
--------------------------
Age:            55
Gender:         Male
Chest Pain:     Asymptomatic
Max Heart Rate: 140 bpm
```

Step 8.2: Convert User Input into Feature Vector

We structure user input into a single-row pandas DataFrame matching the schema of our training data.

In [19]:
```
"""
Purpose of this section:
------------------------
Formats raw user input into a structured DataFrame feature vector.
"""

user_input = pd.DataFrame({
    'Age': [age_input],
    'Gender': [gender_input],
    'ChestPain': [chest_pain_input],
    'MaxHeartRate': [max_heart_rate_input]
})

print("\nUser Input Feature Vector:")
print("==========================\n")
print(user_input)
```

```
User Input Feature Vector:
==========================

   Age Gender     ChestPain  MaxHeartRate
0   55   Male  Asymptomatic           140
```

Step 8.3: Label Encoding of Feature Vector

We encode categorical attributes using the pre-fitted LabelEncoders.

In [20]:
```
"""
Purpose of this section:
------------------------
Applies pre-fitted LabelEncoders to the user input feature vector.
"""

unseen_data_features = user_input.copy()
unseen_data_features['Gender'] = gender_label_encoder.transform(user_input['Gender'
unseen_data_features['ChestPain'] = chest_pain_label_encoder.transform(user_input['

print("\nUser Input Encoded Feature Vector:")
print("==================================\n")
print(unseen_data_features)
```

```
User Input Encoded Feature Vector:
==================================

   Age  Gender  ChestPain  MaxHeartRate
0   55       1          0           140
```

Step 8.4: Load the Saved Model

We load `heart_model.pkl` for inference.

In [21]:
```
"""
Purpose of this section:
------------------------
Loads the saved trained model from heart_model.pkl for inference.
"""

application_model = pickle.load(open('heart_model.pkl', 'rb'))
print("Model loaded successfully for Application Phase!")
```

```
Model loaded successfully for Application Phase!
```

Step 8.5: Model Prediction

We compute the binary prediction and decision probability, formatting the result in a PrettyTable.

In [22]:
```
"""
Purpose of this section:
------------------------
Applies the trained SVC model to the encoded feature vector and displays
```

```
the prediction and confidence in a clean PrettyTable.
"""

predicted_code = application_model.predict(unseen_data_features)[0]
probabilities = application_model.predict_proba(unseen_data_features)[0]

if predicted_code == 1:
    prediction_text = "HEART DISEASE LIKELY"
    risk_level = "Elevated Risk"
else:
    prediction_text = "HEART DISEASE UNLIKELY"
    risk_level = "Low Risk"

disease_probability = probabilities[1] * 100

pretty_table = PrettyTable()
pretty_table.field_names = ["Patient Parameter", "Value"]
pretty_table.add_row(["Age", age_input])
pretty_table.add_row(["Gender", gender_input])
pretty_table.add_row(["Chest Pain Type", chest_pain_input])
pretty_table.add_row(["Max Heart Rate Achieved", f"{max_heart_rate_input} bpm"])
pretty_table.add_row(["Diagnostic Prediction", prediction_text])
pretty_table.add_row(["Disease Probability", f"{disease_probability:.2f}%"])
pretty_table.add_row(["Clinical Assessment", risk_level])

print("\n\n================ DIAGNOSTIC REPORT ================")
print(pretty_table)
```

```
================ DIAGNOSTIC REPORT ================
+--------------------------+----------------------+
|    Patient Parameter     |        Value         |
+--------------------------+----------------------+
|           Age            |          55          |
|          Gender          |         Male         |
|     Chest Pain Type      |     Asymptomatic     |
| Max Heart Rate Achieved  |       140 bpm        |
|  Diagnostic Prediction   | HEART DISEASE LIKELY |
|   Disease Probability    |        76.14%        |
|   Clinical Assessment    |    Elevated Risk     |
+--------------------------+----------------------+
```

Step 9: Execute the Feedback Phase

In real-world health informatics, deployed models require continuous monitoring and clinical validation. Here, we reflect on feedback from domain experts (cardiologists) and identify concrete iterative improvements.

Step 9.1: Collect Feedback from Users and Domain Experts on Performance of the Model Deployed in the Real World

In [23]:
```
"""
Purpose of this section:
-------------------------
```

```
Simulates expert review of the 4-feature model's predictions across representative
"""

expert_feedback = pd.DataFrame({
    "Reviewer": [
        "Cardiologist (Consultant)",
        "Resident Physician",
        "Clinical Data Scientist"
    ],
    "Case / Scenario": [
        "Male, 62 yrs, Asymptomatic chest pain, Max HR 115 bpm -> Predicted DISEASE
        "Female, 40 yrs, Atypical Angina, Max HR 178 bpm -> Predicted DISEASE UNLIK
        "Elderly Patient (75+ yrs) with low max HR -> Predicted DISEASE LIKELY desp
    ],
    "Feedback / Clinical Assessment": [
        "Clinically accurate: elderly males with asymptomatic chest pain and chrono
        "Appropriate: young females with preserved exercise heart rate rarely prese
        "The model heavily weights age and heart rate. Adding ST-depression (oldpea
    ]
})

print(expert_feedback.to_string(index=False))
```

```
                Reviewer
Case / Scenario
Feedback / Clinical Assessment
Cardiologist (Consultant)          Male, 62 yrs, Asymptomatic chest pain, Max HR 11
5 bpm -> Predicted DISEASE LIKELY  Clinically accurate: elderly males with asymptoma
tic chest pain and chronotropic incompetence (low peak HR) have high ischemic risk.
       Resident Physician               Female, 40 yrs, Atypical Angina, Max HR 178
bpm -> Predicted DISEASE UNLIKELY                                    Appropriate: yo
ung females with preserved exercise heart rate rarely present with obstructive CAD.
  Clinical Data Scientist Elderly Patient (75+ yrs) with low max HR -> Predicted DIS
EASE LIKELY despite atypical pain The model heavily weights age and heart rate. Addi
ng ST-depression (oldpeak) and resting blood pressure would reduce false positives.
```

Step 9.2: Make a List of Potential Improvements

In [24]:

```
"""
Purpose of this section:
-------------------------
Compiles actionable engineering and clinical improvements based on feedback.
"""

improvements = [
    "Incorporate ST-depression (oldpeak) and ST-slope to significantly improve sens
    "Add serum cholesterol and resting blood pressure to capture metabolic syndrome
    "Apply cross-validation (5-Fold / 10-Fold CV) to verify stability across divers
    "Compare SVC with non-linear ensemble models like Random Forest and XGBoost for
    "Introduce SMOTE (Synthetic Minority Over-sampling) if training on more severel
]

for idx, item in enumerate(improvements, 1):
    print(f"{idx}. {item}")
```

```
1. Incorporate ST-depression (oldpeak) and ST-slope to significantly improve sensiti
vity for severe ischemia.
2. Add serum cholesterol and resting blood pressure to capture metabolic syndrome in
dicators.
3. Apply cross-validation (5-Fold / 10-Fold CV) to verify stability across diverse d
emographic subgroups.
4. Compare SVC with non-linear ensemble models like Random Forest and XGBoost for no
n-linear feature interactions.
5. Introduce SMOTE (Synthetic Minority Over-sampling) if training on more severely i
mbalanced regional cohorts.
```

Step 9.3: Improve the Model Based on Feedback

In [25]:
```
"""
Purpose of this section:
------------------------
Outlines the operational retraining roadmap for production iterations.
"""

print("Retraining Roadmap for Version 2.0:")
print("-----------------------------------")
print("1. Integrate 'oldpeak' (ST depression) as a 5th clinical feature.")
print("2. Run automated hyperparameter tuning (GridSearchCV over C and gamma).")
print("3. Validate against multi-center cohorts (Hungarian and Long Beach datasets)
print("4. Update 'heart_model.pkl' and deploy seamless zero-downtime update to Stre
```

```
Retraining Roadmap for Version 2.0:
-----------------------------------
1. Integrate 'oldpeak' (ST depression) as a 5th clinical feature.
2. Run automated hyperparameter tuning (GridSearchCV over C and gamma).
3. Validate against multi-center cohorts (Hungarian and Long Beach datasets).
4. Update 'heart_model.pkl' and deploy seamless zero-downtime update to Streamlit Cl
oud.
```

Step 10: Deploy Your Model as a Public Web App

Transforming a trained machine learning model into an accessible web application enables doctors, patients, and instructors to interact with your predictions in real time without touching code.

Step 10.1: Why Deploy a Model?

Machine learning models locked inside local notebooks cannot be evaluated by non-technical stakeholders. Deploying your model via Streamlit allows immediate verification and grading.

Step 10.2: Create a GitHub Account and Repository

1. Log in to your GitHub account at **https://github.com**.
2. Click **+** (top-right) -> **New repository**.
3. Name your repository (e.g., `heart-disease-prediction`), select **Public**, and click **Create repository**.

4. Push your project files to the repository:
```
git init
git add .
git commit -m "Initial commit: Heart Disease Prediction with SVC and Streamlit App"
git branch -M main
git remote add origin https://github.com/<your-username>/heart-disease-prediction.git
git push -u origin main
```

Step 10.3: The Streamlit Web App (`streamlit-app/app.py`)

Our project repository includes `streamlit-app/app.py` and `requirements.txt`:

1. It loads `heart_model.pkl`.
2. It recreates deterministic `LabelEncoder`s matching the notebook categories.
3. It collects inputs using Streamlit widgets (`st.slider`, `st.selectbox`).
4. It calls `model.predict()` and displays the diagnostic risk assessment with calculated probability.

Step 10.4: Create a Streamlit Community Cloud Account

1. Visit **https://share.streamlit.io**.
2. Click **Sign in with GitHub** and authorize Streamlit.

Step 10.5: Deploy Your App

1. Click **Create app** -> **Deploy a public app from GitHub**.
2. Fill in:
   - **Repository:** `<your-username>/heart-disease-prediction`
   - **Branch:** `main`
   - **Main file path:** `streamlit-app/app.py`
3. Click **Deploy**.
4. Streamlit Cloud will install dependencies and launch your application at `https://<your-app-name>.streamlit.app`.

Step 10.6: Troubleshooting Common Issues

- **`ModuleNotFoundError`:** Confirm `requirements.txt` is in the repository with pinned versions. In Streamlit Cloud settings, choose Python **3.11** or **3.12**.
- **`FileNotFoundError: heart_model.pkl`:** Ensure `heart_model.pkl` is committed inside `streamlit-app/` adjacent to `app.py`.
- **App stuck in build loop:** Open **Manage app** (bottom-right of your Streamlit URL), inspect build logs, and trigger **Reboot app**.

Step 10.7: Share Your Live Link

Copy your public `.streamlit.app` URL and paste it into:

1. The very first cell of this notebook (replacing `LIVE_DEMO_LINK_HERE`).
2. Your repository `README.md`.
3. Your Google Classroom assignment submission.

Conclusion

In this project, we successfully converted the reference Titanic Survival Prediction architecture into an end-to-end **Heart Disease Prediction System**:

- Utilizing the UCI Cleveland Heart Disease dataset, we selected **4 clinically vital features** (`Age`, `Gender`, `Chest Pain Type`, `Max Heart Rate`) with zero missing values.
- We implemented systematic preprocessing, label encoding, and an 80/20 train/test split.
- The **Support Vector Classifier (Linear SVC)** achieved **~75.4% test accuracy** and a balanced **0.76 F1-score**, successfully identifying individuals at risk of coronary artery disease.
- The complete pipeline was serialized and deployed as an interactive **Streamlit web application**, accompanied by domain expert feedback and a clear clinical retraining roadmap.